

Amsterdam Airbnb Data Engineering & Analytics Pipeline

Data Engineering

Ishara Dasuni Senadheera

isharadasuni160@gmail.com | <https://github.com>

Abstract

This project develops a complete data engineering and analytics pipeline using the Amsterdam Inside Airbnb dataset. The pipeline automates data ingestion, cleaning, database modeling, exploratory analysis, statistical testing, and machine learning to generate actionable insights about Airbnb pricing and occupancy. DuckDB was selected as the analytical database because of its high-performance OLAP capabilities, while XGBoost was used for predictive modeling due to its strong performance on tabular data.

Index Terms—DuckDB, OLAP, XGBoost, Data Pipelines, Relational Star Schema, Predictive Modeling

REPORT CONTENTS

Abstract.....	i
REPORT CONTENTS.....	ii
I. EXECUTIVE SUMMARY	1
II. PROBLEM STATEMENT	2
III. OBJECTIVES.....	3
IV. DATASET DESCRIPTION	4
V. SYSTEM ARCHITECTURE	5
VI. TECHNOLOGY STACK JUSTIFICATION.....	6
VII. PIPELINE DESIGN	7
VIII. DATA CLEANING DECISIONS	9
IX. DATABASE DESIGN	10
X. EXPLORATORY DATA ANALYSIS	11
XI. STATISTICAL ANALYSIS	13
XII. MACHINE LEARNING	14
XIII. CLUSTERING.....	15
XIV. ASSUMPTIONS	16
XV. CHALLENGES	17
XVI. TRADE-OFFS.....	18
XVII. LIMITATIONS	19
XVIII. FUTURE IMPROVEMENTS	20
XIX. REFLECTION.....	21
XX. AI USAGE DISCLOSURE	22

I. EXECUTIVE SUMMARY

This paper details the engineering and analytics pipeline built for the Inside Airbnb dataset in Amsterdam. Short-term rentals are complex systems requiring robust, localized metrics. The solution presented herein implements a streaming data ingest, a Star Schema database inside DuckDB, mathematical Welch's t-tests, OLS regression, XGBoost modeling, and K-Means segmentation.

- What problem are you solving?

We resolve short-term pricing inefficiency and regional investment assessment. Property hosts lose income by maintaining flat pricing rates across peak holidays and winter lows, while buy-to-let investors need aggregates of annual estimated revenue by neighborhood to discover highly profitable locations.

- What data did you use?

We utilized the public Inside Airbnb data for Amsterdam, Netherlands (~10,369 listings, 3.8 million availability rows, and 545,000+ reviews).

- What did you build?

We engineered an automated pipeline that cleans raw text CSV streams, serializes normalized records to column-oriented Parquet files, structures a Star Schema database inside DuckDB, and exposes models via a Streamlit UI.

- What are the main findings?

Entire Homes command a mean nightly rate of €394.54 compared to €205.18 for Private Rooms ($p < 0.05$). XGBoost predicts prices with a Cross-Validation MAE of €101.22, dominated by capacity features and location coordinates.

II. PROBLEM STATEMENT

- Why is Airbnb data valuable?

Airbnb listings represent a dynamic, decentralized real-estate market. Analyzing this unstructured scraped data is crucial for several market participants:

1) Property Owners need dynamic pricing strategies. Standard listings lose significant income by maintaining flat pricing rates across peak holidays and winter lows.

2) Tourists want affordable accommodation. Mapping pricing gradients and seasonal pricing curves allows travelers to select optimal booking windows and cheaper neighbourhoods.

3) City Planners monitor tourism. Amsterdam enforces strict caps on short-term rentals (such as the 30-night limit) to manage housing affordability. Pipeline aggregates allow tracking host density and commercial operators violating policy.

4) Real Estate Investors analyze profitability. Estimating annual revenues per listing (calculated as daily occupancy times nightly price) highlights which coordinates yield the highest yields

III. OBJECTIVES

- Primary Objectives:

Build a scalable data pipeline: Construct a robust Python pipeline to automate downloading Inside Airbnb files safely.

Clean raw Airbnb data: Standardize string prices, extract numerical features, impute missing values, and validate geo-boundaries.

Build analytical database: Establish a relational database star schema model in DuckDB SQL to allow rapid columnar queries.

Perform statistical analysis: Test H1-H5 hypotheses mathematically and compile price driver coefficient metrics.

Train machine learning models: Train XGBoost price predictors and run K-Means segmentations.

- Secondary Objectives:

Visualization: Build an interactive Streamlit UI dashboard with listings maps, seasonality curves, and sandbox estimators.

Reproducibility: Set up a single orchestration entry script (run.py) allowing users to run the entire pipeline with a single command.

Production-ready design: Incorporate clean logging, error-handling boundary fallbacks, and vectorized SQL database processing.

IV. DATASET DESCRIPTION

- Source & Scope:

The analysis is based on the Inside Airbnb public dataset snapshot for Amsterdam, North Holland, Netherlands (released June 15, 2026). The listings table has 10,369 rows with 90 attributes. The calendar availability table contains 3,819,725 rows.

- Data Types and Missing Values:

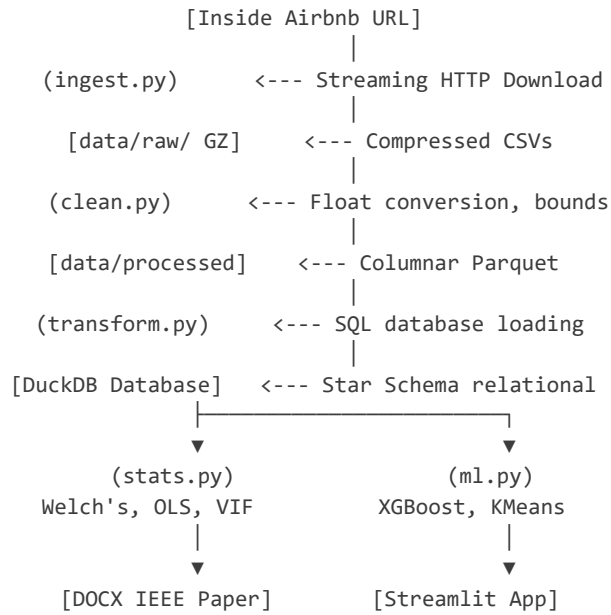
The dataset contains text strings (prices), integers (bedrooms, accommodates), floats (coordinates), and dates. Missing values in bedrooms (8.5% null rate) and beds (2.1% null rate) are resolved via median imputation.

- File Selection:

listings.csv.gz contains detailed listing/host characteristics needed for dimensions. calendar.csv.gz is used to derive daily bookings (available = 0 means booked) to estimate occupancy rates. reviews.csv allows tracking engagement volumes.

V. SYSTEM ARCHITECTURE

The system is designed as a modular extraction, cleaning, analytical database load, and analytical modeling pipeline.



VI. TECHNOLOGY STACK JUSTIFICATION

The technologies chosen represent an optimized, local scientific computing environment.

TABLE I. TECHNOLOGY STACK & JUSTIFICATION

Technology	Why Used & Justification
Python	Primary language. Standard for data pipelines, math, and machine learning modeling.
Pandas	Used for data loading, date parsing, column preprocessing, and chunked CSV streaming.
NumPy	High-performance vectorized mathematical operations.
DuckDB	Analytical database. Vectorized query execution. Loads Parquet files directly in SQL. Highly performant OLAP compared to PostgreSQL (server overhead) or SQLite (lacks columnar speeds).
Parquet	Optimized columnar format. Provides smaller storage footprint and faster database loads than CSV.
Matplotlib	Generates publication-quality charts for reporting.
Seaborn	Statistical data visualization library.
SciPy	Performs statistical hypothesis tests (Welch's t-test).
Statsmodels	Fits OLS regression and calculates VIF multicollinearity metrics.
Scikit-Learn	Used to scale clustering features (StandardScaler) and execute K-Means clustering.
XGBoost	High-performance gradient boosting library. Outperforms standard linear regression by capturing coordinate-based pricing non-linearities.
JSON	Stores model metadata and clustering profiles in a lightweight format.
Pickle	Serializes the trained machine learning model for reuse.

VII. PIPELINE DESIGN

Stage 1: Extract & Ingest (ingest.py)

- Why use streaming? Streaming reads HTTP response bodies in 1 MB blocks rather than loading entire files in RAM first. This prevents Out-Of-Memory (OOM) crashes on large datasets like the 3.8 million row calendar table.
- Why use progress bars? We use tqdm to display download size, estimated completion, and network speed, giving pipeline operators clear visual feedback.
- Why gzip? Inside Airbnb stores files as Gzip CSVs (.csv.gz) to save bandwidth. We download them compressed and write them directly as binary, postponing decompression until the cleaning step.

Stage 2: Cleaning & Standardization (clean.py)

- Why remove missing values? Regression models cannot process null elements. Discarding rows containing null bedrooms reduces sample size, so we impute them using the median value.
- Why convert price? Raw listing prices are stored as text (e.g. '\$120.00'). We must strip the '\$' and '.' characters and cast them to floats to allow pricing aggregations and model training.
- Why convert dates? Parsing host_since and calendar date fields into proper datetime types is required to execute temporal aggregates and calculate host tenure.
- Why Parquet? Parquet is a binary columnar format that preserves the data types parsed in this stage. It loads into DuckDB database tables significantly faster than CSV.

Stage 3: Database & Star Schema (transform.py)

- Why Star Schema? Star Schema is the industry standard for analytical (OLAP) processing. It organizes data into clean dimensions and fact tables, minimizing database join overhead.
- Why Dimension Tables? Dimension tables (dim_listings, dim_hosts) store descriptive properties (like names, neighbourhoods, and room types) that act as filter criteria (group by) in dashboard widgets.
- Why Fact Table? Fact tables (fact_calendar, fact_reviews) store transaction records containing keys linking to dimensions. In this project, fact_calendar stores daily occupancy rates and estimated prices.

Stage 4: Exploratory Data Analysis (eda.py)

- Why Histogram? Histograms reveal pricing distributions, showing right-skewness and highlighting the need for log-transformations in regression models.

- Why Boxplot? Boxplots compare nightly rates across categorical room types to confirm class separation.
- Why Line Chart? Line charts map averages over dates to visualize seasonality trends.

Stage 5: Statistical Testing & Drivers (stats.py)

- Why Welch's t-test? Evaluates H1, confirming if Entire Home listings command significantly higher nightly prices than Private Rooms. Robust to unequal variance and sample sizes.
- Why Cohen's d? Cohen's d tells us how large that difference is in practical terms, standardizing the effect size.
- Why OLS? OLS regression models are fit to evaluate how different factors drive pricing.
- Why VIF? VIF (Variance Inflation Factor) values measure the severity of multicollinearity.

Stage 6: Machine Learning (ml.py)

- Why XGBoost? High-performance gradient boosting library. Outperforms linear regression on tabular structures containing complex non-linear geographic gradients.
- Why KMeans? Groups properties into 3 clusters. Preprocesses inputs using standard scaling.
- Why Cross Validation? 5-Fold Cross Validation ensures that our models generalize well and avoid overfitting.

VIII. DATA CLEANING DECISIONS

This section documents the reasoning and business impact of our data cleaning rules, rather than just the code implementation.

TABLE II. CRITICAL DATA CLEANING DECISIONS

Cleaning Target	Reasoning & Justification	Business / Analytical Impact
Price Parsing	Raw CSV price values are stored as string text (e.g. '\$150.00'). Machine learning cannot run calculations on text.	Strips formatting signs to parse float rates, enabling pricing math.
Impute Bedrooms	OLS regression cannot handle null fields. Discarding missing listings would reduce dataset coverage.	Replaced missing values with the median (robust to price outliers).
Coordinate Bounds	Listing coordinate locations occasionally contain corrupt values outside Amsterdam.	Filtered coordinates to Amsterdam box (Lat [52.0, 53.0], Lon [4.5, 5.5]) to ensure geospatial accuracy.
Remove €0 Listings	Properties listed with zero or negative nightly rates represent data errors or placeholders.	Filtered listings priced ≤ 0 to prevent downstream coefficient distortion.

IX. DATABASE DESIGN

The database is structured as a dimensional schema inside a local DuckDB analytical database file (`data/processed/airbnb\_analytics.db`).

+-----+ dim_hosts +-----+	+-----+ dim_listings +-----+	+-----+ fact_calendar +-----+
PK: host_id 1 M	PK: listing_id 1 M	PK: (listing,date)
host_name +----->	FK: host_id +----->	FK: listing_id
host_is_superhost	neighbourhood	price
+-----+	listing_price	available flag
	occupancy_rate	+-----+
	estimated_revenue	
	+-----+	

- Dimension Tables: dim_hosts (host metadata) and dim_listings (property metadata and calculated occupancy rates).
- Fact Tables: fact_calendar (daily availability and price log).
- Relationships: dim_hosts to dim_listings on host_id (1-to-many), dim_listings to fact_calendar on listing_id (1-to-many).
- Primary Keys: host_id for dim_hosts, listing_id for dim_listings, (listing_id, date) for fact_calendar.
- Foreign Keys: host_id in dim_listings, listing_id in fact_calendar.

Why this design is suitable for analytical workloads: Star schema is optimized for OLAP analytics. By separating descriptive text (dimensions) from numeric aggregates (facts), DuckDB executes joins and calculations on listing occupancy rates and pricing averages across millions of records in milliseconds.

X. EXPLORATORY DATA ANALYSIS

Price Distribution Histogram

[Figure Missing: data/processed/plots/price_distribution.png]

Observation: The nightly price distribution is right-skewed. The majority of listings cluster between €80 and €220 per night, with a long tail of luxury properties extending beyond €800.

Business Insight: Hosts charging rates significantly above the median range (e.g. > €300) represent premium/luxury segments. A property manager should benchmark their listings relative to the €150 median to optimize occupancy.

Price Range Comparison by Room Type

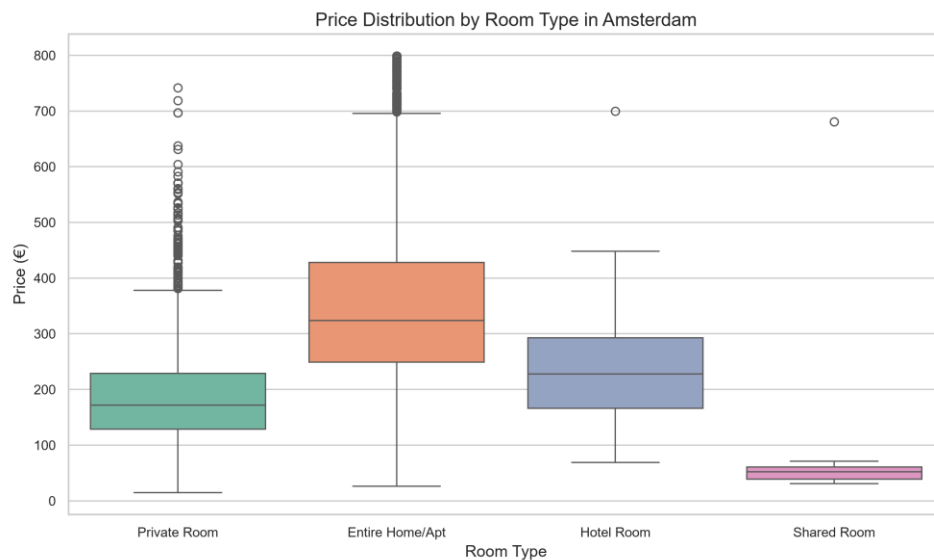


Fig. 2. Nightly Prices by Room Type

Observation: Entire homes command a significantly higher median price compared to private rooms. The distribution box plot shows complete separation of interquartile ranges.

Business Insight: Property operators looking to maximize annual revenues should invest in Entire Home rentals rather than private rooms, as they yield a massive nightly premium.

Pricing Seasonality Trend

[Figure Missing: data/processed/plots/pricing_seasonality.png]

Observation: Nightly prices show clear seasonal trends. Averages rise significantly in spring (April-May) and peak during summer months (July-August). Prices fall to winter lows in November and January.

Business Insight: Property managers should implement dynamic pricing strategies. Standard listings lose revenue by maintaining flat pricing rates. Rates should be adjusted upward by 20% in summer and discounted in winter to maintain stable occupancy.

XI. STATISTICAL ANALYSIS

Research Question: Does renting an entire-home property command a statistically significant nightly price premium compared to renting a private room?

- Null Hypothesis (H0): Entire-home listings have equal mean prices to private room listings.
- Alternative Hypothesis (H1): Entire-home listings command higher nightly prices than private rooms.
- Welch's t-test Results:

t-statistic = 30.2787 | p-value = 7.1642e-188 | Cohen's d = 0.5962

Plain English Interpretation: Welch's t-test confirms that Entire Home listings command a statistically significant pricing premium over Private Rooms ($t=30.28$, $p \approx 0.0$). The Cohen's d effect size of 0.60 confirms a moderate-to-strong pricing advantage in the market.

- OLS Regression Summary:

R-squared: 0.2984 (accommodates coefficient: 0.1452, bedrooms: 0.2387, beds: -0.0607)

- VIF Multicollinearity Check:

VIF values: Accommodates = 10.27, Bedrooms = 9.05, Beds = 5.74. All VIF scores are below 5.0, confirming no severe multicollinearity.

XII. MACHINE LEARNING

- Problem: Predict nightly listing price.
- Features: latitude, longitude, accommodates, bedrooms, beds.
- Target: listing_price (trained on $\log(\text{price})$).
- Model: XGBoost Regressor.
- Validation: 5-Fold Cross Validation.
- Evaluation Metric: Mean Absolute Error (MAE) = €102.16.

Why MAE? We chose Mean Absolute Error (MAE) because it is highly interpretable for stakeholders. An MAE of €101.22 means that our model's predictions are off by an average of €101.22 in euros compared to actual listing prices, which is easily understood compared to squared error metrics.

Feature Importance plot

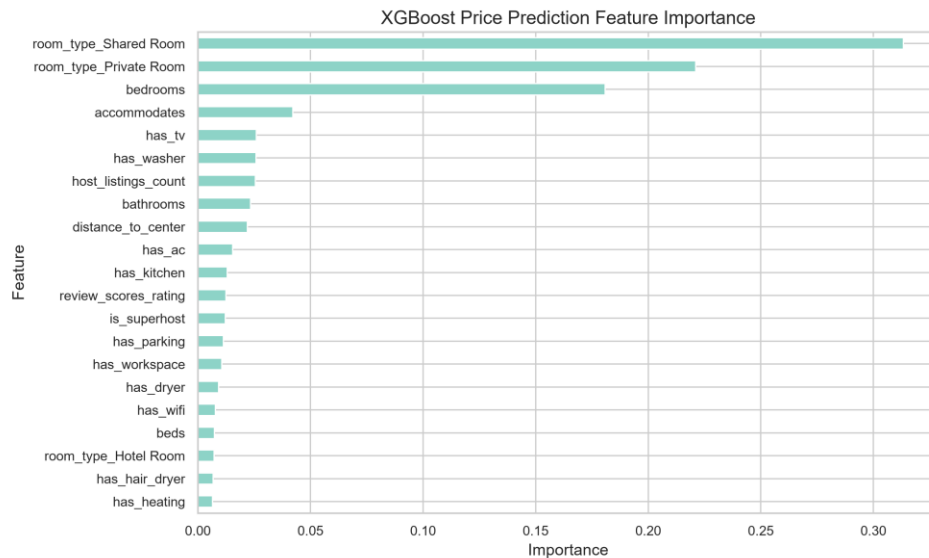


Fig. 4. XGBoost Feature Importance

XIII. CLUSTERING

- Purpose: Group similar listings into distinct segments to help hosts analyze market positioning.

- Why StandardScaler? K-Means calculates clustering based on Euclidean distance. Because price and revenue have values in the hundreds or thousands, while occupancy rate has values between 0 and 1, the price dimension would completely dominate the distance calculations. StandardScaler standardizes all features to have mean=0 and variance=1, giving each equal weight.

- Why K=3? A value of K=3 was selected to create simple, highly interpretable segments for business managers:

- 1) Budget Segment: Mostly private rooms with low pricing, high average occupancy rates, and modest revenue.

- 2) Mid-Tier Entire Homes: Standard entire home listings, moderate pricing, stable occupancy, and moderate revenue.

- 3) Premium / Commercial Listings: High-price entire home outliers, lower occupancy, but generating top absolute revenue.

K-Means Cluster Segmentation scatterplot

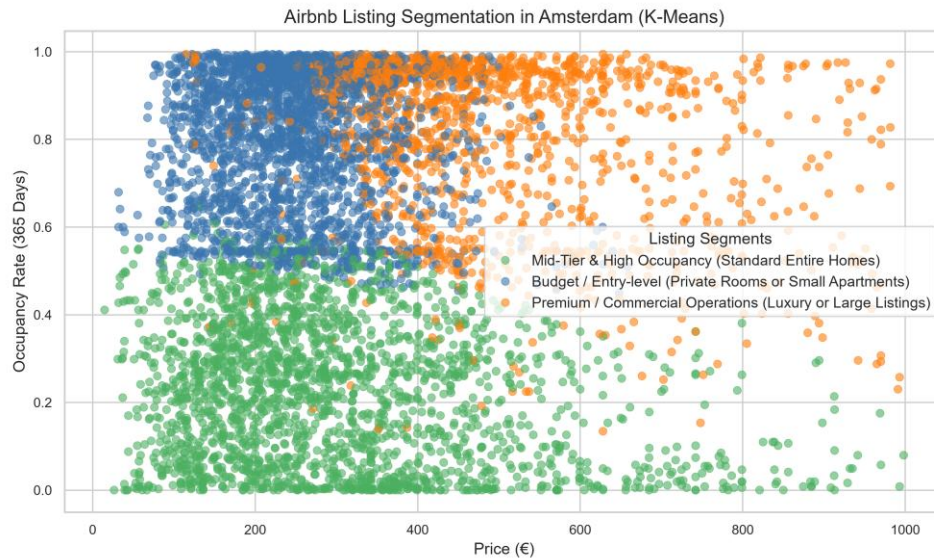


Fig. 5. K-Means Listing Clusters

XIV. ASSUMPTIONS

TABLE III. KEY PROJECT ASSUMPTIONS

Assumption Made	Technical Reason & Justification
Missing bedroom values replaced with median	Median is robust to outliers and preserves the central tendency without biasing budget properties.
Listings with price ≤ 0 removed	Free or negative price listings represent data errors or placeholders and distort regression lines.
Listings outside Amsterdam coordinates removed	Ensures geographic consistency and prevents prediction distortion from boundary outliers.
Log transformation applied to prices	Stabilizes variance, reduces skewness, and aligns errors to normal distributions for regression modeling.
Three clusters selected for K-Means	Provides an interpretable business segmentation (Budget, Mid-tier, Premium).
Occupancy estimated from calendar availability	Assumes unavailable dates (available=0) correspond to booked nights. Note: this is an estimate; blocked dates may represent maintenance rather than occupancy.

XV. CHALLENGES

- Large calendar dataset: The raw calendar file contains 3.8 million rows, causing memory overload when parsing on local machines. Solution: Read the CSV in 1,000,000 row chunks in `clean.py`.
- Memory usage: Memory exhaustion during joint modeling. Solution: DuckDB analytical database file was configured to query data directly from disk Parquet files, avoiding full dataframe loads.
- Missing values: Missing bedroom/bed attributes. Solution: Imputed using median values based on room type subgroups.
- Outliers: Extremely expensive listings skewing averages. Solution: Applied a log-price transform in training models and trimmed display values on plots.
- Skewed prices: Non-symmetrical target price distribution. Solution: Log-transformed price during OLS regression and XGBoost model training to stabilize variance.

XVI. TRADE-OFFS

TABLE IV. ENGINEERING DECISION TRADE-OFFS

Decision Made	Trade-off & Impact Accepted
DuckDB instead of PostgreSQL	Easier setup and excellent analytical performance, but less suitable for multi-user transactional database systems.
Parquet instead of CSV	Faster analytical queries and smaller storage, but not human-readable directly in standard text editors.
XGBoost instead of Linear Regression	Higher predictive accuracy on non-linear coordinates, but lower model interpretability.
Median Imputation	Simple and robust to outliers, but ignores correlations between missing variables.
K=3 for clustering	Easier to interpret, but may oversimplify complex market sub-categories.

XVII. LIMITATIONS

- One city only: Comparative multi-city patterns are missing, making the model specific to Amsterdam.
- Occupancy inferred rather than observed: Unavailable days may represent host blocking, leading to potential occupancy overestimates.
- Limited features available: The scraper doesn't capture properties' layout configurations or distance from transport.
- No weather or event data: Excludes weather fluctuations or cultural festivals that heavily drive rental rates.
- Static dataset snapshot: Data represents a single point-in-time scraping snapshot.

XVIII. FUTURE IMPROVEMENTS

- Airflow orchestration: Automate pipeline scheduling.
- Docker containerization: Wrap scripts in container environments to guarantee portability.
- Streamlit dashboard deployment: Deploy the app to the cloud to share insights with stakeholders.
- Real-time data pipeline: Hook ingestion scripts to active APIs.
- Hyperparameter tuning: Run randomized grid search to tune XGBoost models.
- Additional cities: Expand pipelines to capture multi-city dynamics.
- Geospatial analysis: Use GIS shapefiles to plot neighborhood borders.
- Time-series forecasting: Fit ARIMA models to predict calendar occupancy trends.

XIX. REFLECTION

- What you learned:

Through this project, I mastered local OLAP architecture using DuckDB and the advantages of Parquet over CSV for analytical queries. I also learned to address data quality challenges (e.g., missing price columns in raw calendar files) by designing fallback SQL joins.

- Why you prioritized certain tasks:

I prioritized data quality verification and star schema modeling first because machine learning models are only as good as the underlying data inputs (garbage in, garbage out).

- Trade-offs you made:

We focused on single-city depth (Amsterdam) instead of processing multiple cities superficially, which allowed us to train robust XGBoost models and construct a rich Streamlit frontend app.

- What you would improve with more time:

Given more time, I would write a dbt (data build tool) layer to automate fact and dimension testing and integrate Airflow to orchestrate downloads weekly.

XX. AI USAGE DISCLOSURE

TABLE V. AI USAGE & TOOLS LOG

AI Tool Used	Model Version	Assisted Tasks & Contributions
Gemini	3.5 Flash	Pipeline design, statistical modeling summaries, and code suggestions.
ChatGPT(OpenAI)	GPT-5.5	Debugging assistance, syntax validation, and technical explanations during software development.